Zaccery Tarver

10.7.2025

Operating Systems Analysis

Honors Credit by Contract Report

Introduction to NixOS

When it comes to Linux distributions, we are familiar with Ubuntu and Debian through its popular derivatives like Linux Mint and Raspberry Pi OS. We may overlook other Linux Distributions such has Red Hat Linux, OpenSuse, and Arch Linux, which are admittedly geared towards more experienced users and system administrators. NixOS is one of these systems. As someone who has been experimenting with Linux since 2023, I have become quite familiar with OS installations, using the bash shell, and exploring open-source alternatives. NixOS is definitely uncommon, but a rather unique operating system. It keeps the entire system configuration in one file so that it is reproducible. Program versions are kept track of by their cryptographic hash. NixOS takes a different approach to user and system management, and this paper will be an introduction of what NixOS has to offer.

1.0 Installation

I will be installing the minimal ISO image, which does not contain a graphical user interface. As someone who has completed several Linux Mint installations, I will be challenging myself with a non-GUI approach. This also gives me the option for what desktop environment I would like to use, as opposed to being given one by default in the live ISO. Most importantly, however, I can focus on core features of NixOS that make it unique, instead of blending it with GUI utilities that are common in all distributions. I will be installing the 64-bit Intel/AMD ISO, since my target computer has the x86 CPU architecture.

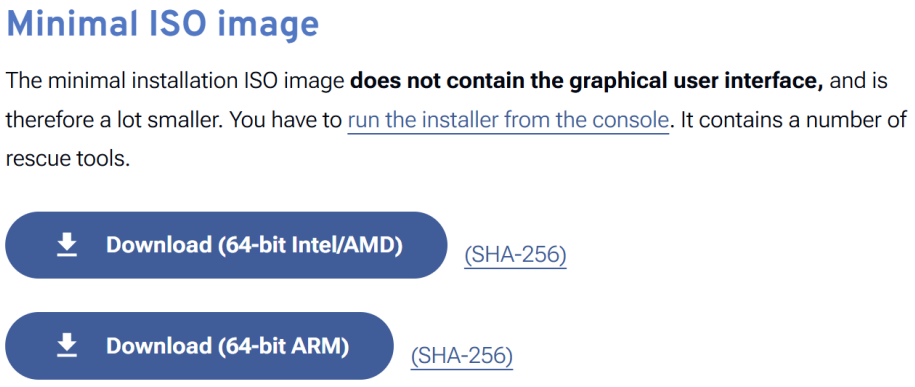

Figure 1.0.A NixOS Install Page

1.1 Verifying Download and Creating Boot Medium

I will be using a Windows 11 PC to verify the install and to create the bootable USB. Verifying any installation can be done by comparing the SHA256 hash of the downloaded file with the hash prior to download, provided by an existing plain text file. I run these commands in PowerShell:

- certUtil -hashfile {my_file} SHA256
- '{hash_from above command}' -eq '{hash_from text_file online}'

These commands obtain the hash from the downloaded iso and then compare it with the hash prior to transit. When the second command returns true, it means that the hashes are equal, and the file passes the integrity check. The first command can be found from Apache documentation, and the second command can be found from Microsoft PowerShell documentation.

I will be using Balena Etcher to write the iso image to the USB drive.

1.2 Booting the Installation Media

I will be installing NixOS on an older laptop I have. With the bootable media, I first needed to disable Secure Boot in the BIOS settings of this laptop. Secure Boot interferes with the ability for the bootable media to properly load (likely because it is an uncommon operating system). We also don't need Secure Boot because we have already performed an integrity check on our ISO.

At the installer screen, I chose the Long-Term Support version, to ensure security updates from the NixOS developers.

Once I get to the Live ISO terminal, I run the command nixos-help to get a comprehensive guide on the operating system. My first task is installing NixOS to my computer's hard drive.

1.3 Installation Summary

The NixOS Installation was a relatively straightforward but technical process. I used the nixos-help command to display the documentation on Manual NixOS Installation. Since the goal of this paper is to cover the uniqueness of the Operating System itself, I will not be describing the full installation process, since that will end up being just a summary. But here are some key steps below.

First, I set up my network with the wpa_supplicant utility:

$ sudo systemctl start wpa-supplicant

$ wpa-cli

The network is used to obtain resources needed for installation. Next, I use the GNU parted command utility to partition my device HDD, allocating space for the main ext4 file system, and swap partition (for RAM substitution). Then, I formatted these partitions, to ready them for installation. Then, I setup default system configurations (covered in the next section), and ran nixos-install. Finally, after running nixos-reboot, my system was ready to use NixOS.

2.0 NixOs – The Declarative Operating System

What makes NixOS so unique is that it is declarative. That is, the entire system configuration lives in a file /etc/nixos/configuration.nix. When installing the system in the previous section, the final step before running nixos-install was to modify this configuration file for the purposes seen fit. This file can now be modified at any time to change the system behavior.

2.1 Editing /etc/nixos/configuration.nix

The configuration file can easily be modified by running:

```
$ sudo nano /etc/nixos/configuration.nix
```

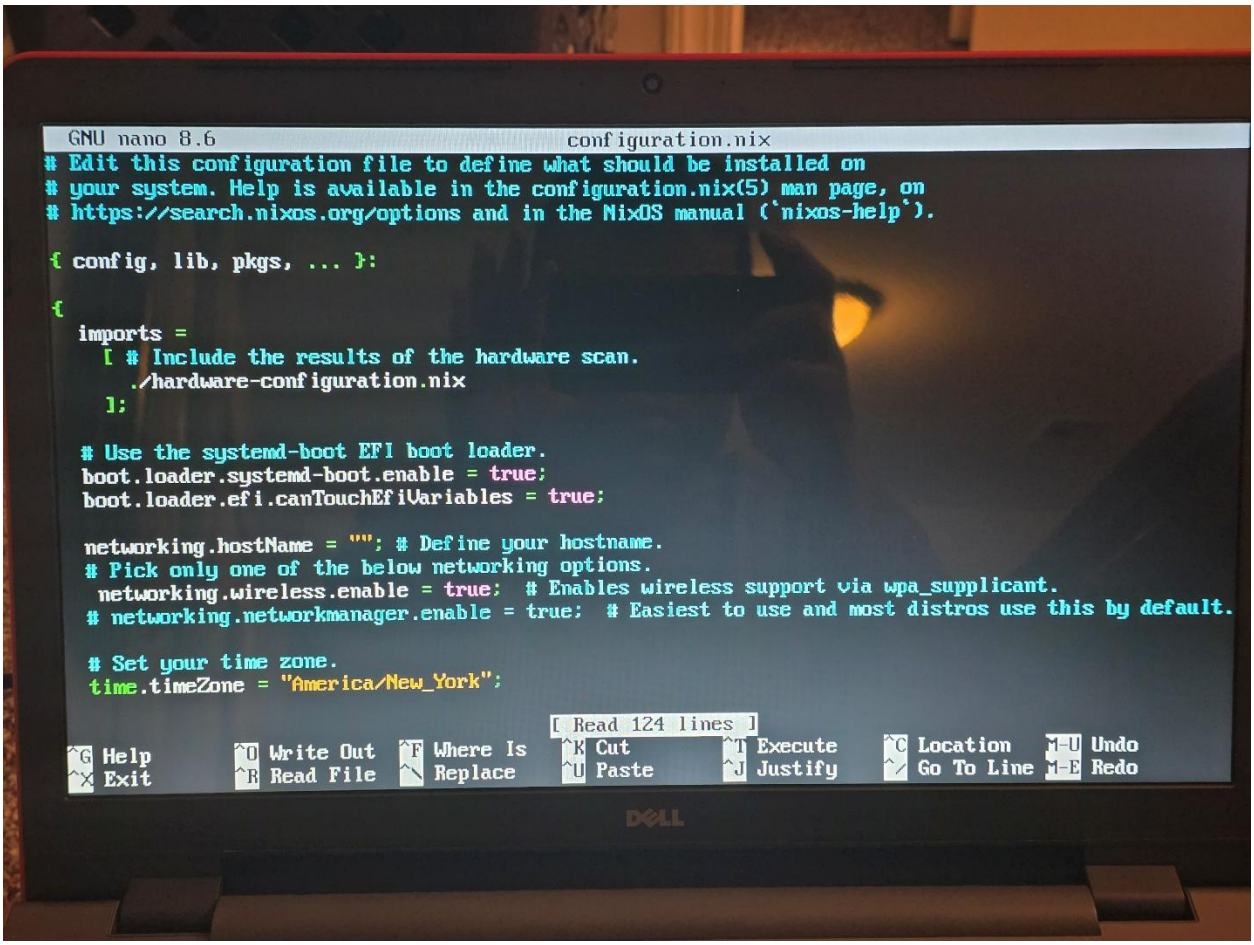

Figure 2.1.A Here is a look at the configuration.nix file. We can configure the system in various ways, i.e. the time zone.

2.1.1 Basic Expression Syntax and Modularity

The file is referred to as a Nix expression. The Nix expression adheres to the structure of the Nix language. For example, the first line of the configuration file is

{config, pkgs, ... }:

Which describes a function that handles system configuration, packages, etc.

In the configuration file, we can modify various system services and packages to change the way we use our NixOS device. For example, one may enable the Hyper Text Transfer Protocol Daemon (httpd) with the following line:

services.httpd.enable = true;

This statement is called a Nix option definition, and can alternatively be written as:

services = {

```
httpd {
        enable = true;
}}
```

This allows us to modularize our approach to Nix options, which is handy when defining multiple services, or multiple attributes of httpd, all at once. We can take the modularity a step further by dividing our Nix option definitions into separate files, and then importing each of these files into configuration.nix. For example:

```
Imports = [
./vpn.nix
./kde.nix
];
```

Can be used to configure our Virtual Private Network or Desktop Environment in separate files.

To rebuild the system based off a modified configuration.nix file, we must use the command:

```
$ nixos-rebuild switch
```

To rebuild the system and switch to this new configuration. What's great about nix is that we can choose previous system configurations on the GRUB boot menu before we load the OS, meaning we can rollback from undesirable changes and effectively reproduce our system setup by transferring our configuration.nix file to any other NixOS system.

2.1.2 Package Management

When editing the Nix configuration, it is essential to add software packages to the user environment that will help us use our system more effectively. We can add packages to our system with the following line to configuration.nix:

```
environment.systemPackages =
        [ pkgs.thunderbird
          pkgs.emacs
        ];
```

By modifying the environment.systemPackages option, we can add software to the system like Mozilla's Thunderbird email client and the popular emacs editor. Of course, we must keep in mind that any packages with GUIs will not work on a system with no desktop environment, which would require separate configuration to make work.

After running nixos-rebuild switch, any packages that we have added to our configuration will now become usable on the system.

The method shown previously demonstrates declarative package management. But we can also use ad-hoc package management. This is more like traditional package managers like apt on Ubuntu systems. For example,

nix-env -iA nixos.firefox

will install Firefox much like this counterpart command for Ubuntu systems:

sudo apt install firefox

We can upgrade all packages by running

nixos-env -u '*'

We can remove packages by running

nix-env -e thunderbird

And even rollback our nix environment if we make undesirable changes to our system

nix-env --rollback

And of course, just like in other operating systems, we can download an archive file online and build the software from source. When combined with NixOS's unique and traditional methods of package management, any individual has a lot of control over what software runs on their system.

NixOS also offers an experimental, unstable feature called flakes. Flakes allows the user to write Nix expressions that are dependent on respective application versions, which allows more granular control of the system. However, until this feature is deemed non-experimental and therefore stable, I recommend using declarative or ad-hoc methods of package and dependency management.

3.0 Filesystem

NixOS's file system is very similar to other Linux distribution file systems such as the rockhopper/plato servers we access in class.

Nix OS uses the Bourne Again Shell (Bash) as its default shell to control the system. The bash shell can be used through TTY, or the TeleTYpewriter, the default (and only) terminal interface included in the minimal version of NixOS. With the bash shell, we can set our current directory to root, and then view the contents of the system:

$ cd / ; ls

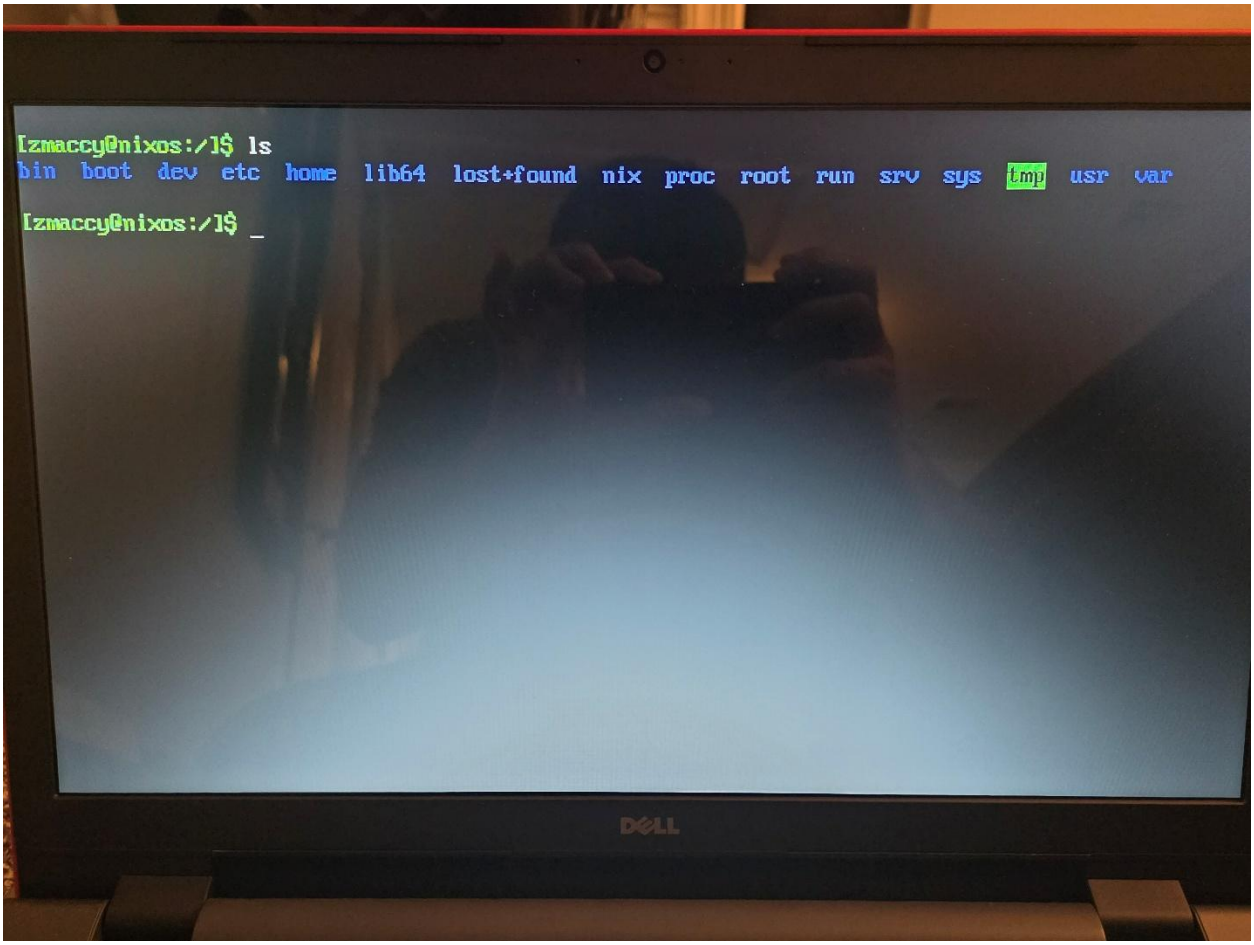

Figure 3.0.A The subdirectories of root, which are all pretty standard in Linux filesystems.

The subdirectories are generally the same as on other Linux systems. It is worth mentioning the proc subdirectory which we have worked with in class and on certain assignments with. There is also the nix directory, which contains the store folder, which is the nix store. The etc folder contains the /nixos/configuration.nix file, and configuration for other services like wpa_supplicant.

3.1 The Nix Store

Any packages obtained through declarative or ad-hoc methods are located within the nix store. The Nix store is located at: /nixos/store. The Nix store organizes packages based off of cryptographic hashes calculated from package data such as the package's dependencies. This means that packages, even versions of the same package, are kept in different directories. This allows the Nix system to remain modular. For example, older system configurations may use an older package in the Nix store, while newer configurations may use a newer package in the Nix store. This allows packages to remain purely functional, meaning that they do not interfere with each other's operability. So debugging package problems in the nix environment should be ideally very straightforward.

4.0 My Experience with NixOS

In this final section, I will describe my personal experience using NixOS and conclude the paper, discussing potential future routes of research.

Prior to installation, I setup my configuration.nix file to include my user account, using the template generated during installation setup:

```
users.users.zmaccy = {
    isNormalUser = true;
    extraGroups = [“wheel”];
    packages = with pkgs;[
        tree
    ];
];
```

Being in the wheel group allows my account to use the sudo command to grant super user privileges.

The first thing I did with the newly installed NixOS was set up a network connection. To do this, I set the nix option networking.hostName to the empty string. By doing so, the hostname will be resolved through Dynamic Host Configuration Protocol (DHCP). I also enabled the services.openssh.enable option to true, so that my system can be used as an ssh client. After this, I re-built the system and switched to this new configuration.

To continue with my network test, I setup the file /etc/wpa_supplicant.conf file. The acceptable formats of this file can be found at: wpa_supplicant.conf(5) - Linux man page. I follow the format for WPA-Personal home networks. After configuring my WiFi, I test the new network connection with ssh, and I am able to login to rockhopper.monmouth.edu from this device! I thought this was pretty impressive.

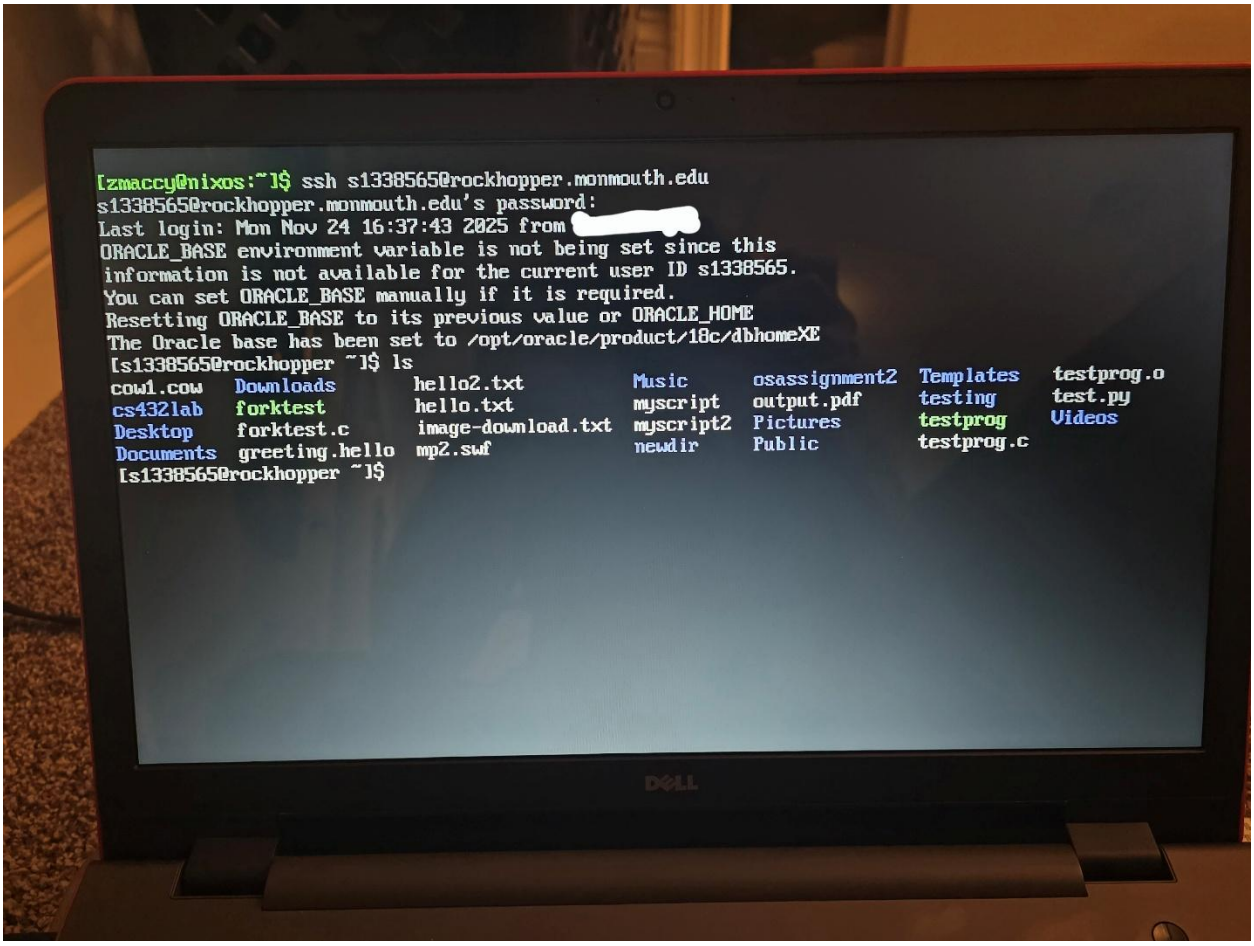

Figure 4.0.A OpenSSH client connected to the host rockopper.monmouth.edu

With a network connection, I could now install packages. I edited the environment.systemPackages option to include pkgs.fish. Fish stands for the friendly interactive shell and is another shell we can use to control the system. Fish supports syntax highlighting of commands, command suggestions based on history, and legacy terminal color output. It also has very user-friendly documentation

After running nixos-rebuild switch, Fish is installed and ready to use. Fish works excellently in the terminal and is a quality-of-life improvement for any avid terminal user. Since it supports most of bash's commands it can be used as a replacement for the default NixOS shell.

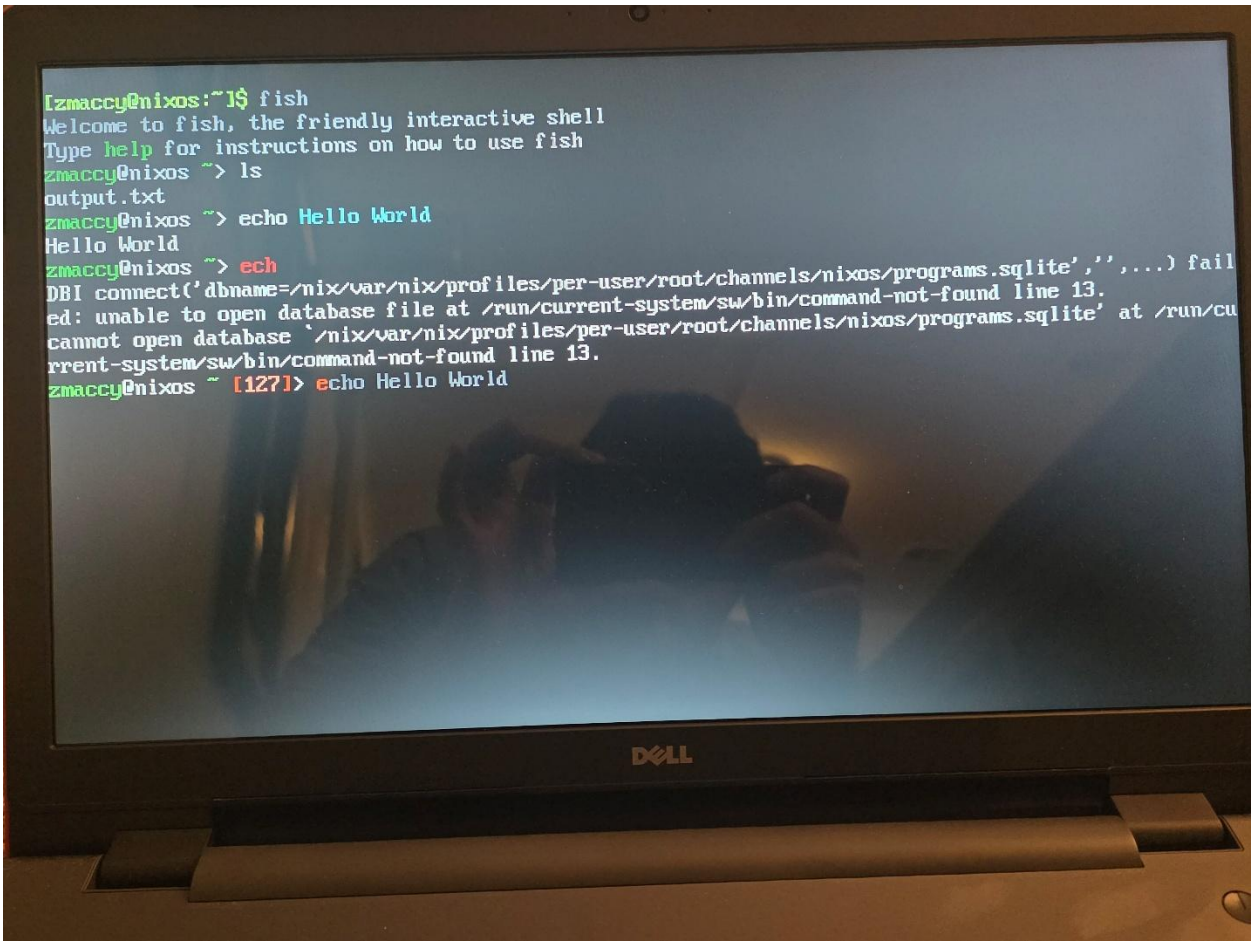

Figure 4.0.B Sample Fish interaction. We can use bash commands like echo. If a command is not valid, an error message is displayed to the screen showing that there is no such program on the system that is referred to by the attempted command. Fish highlights illegal commands in red by default, and will even suggest a command based off the history of commands, which can be seen in the last line of the image shown.

4.1 Conclusion and Future Work

In this report, I covered Installation, Nix Configuration, Package Management, the File System, and my personal applications of the documentation I researched. This was a challenging process, especially installation: the minimal terminal interface was very technical to use. But despite the difficulty, I really appreciated exploring a more foundational and manual approach to OS system installation and configuration. This gives us greater control over the system, and allows us to design it for exactly what we need. In contrast to buying a PC with a ton of pre-installed features, the methods explored in this paper are more challenging, but effectively cut down on the bloatware of modern proprietary operating systems.

For future work, I can download more packages for programming environments, like java, python, gcc, etc. I can explore the customization of the alternative Fish shell for user personalization and optimal system control. I can also set up a desktop environment so that I can use everyday tools like Firefox. I can also configure my OS with proper drivers to view optical disc media, and interface with USB devices like drives and game controllers.

References

[1]

sdwheeler, "about_Comparison_Operators - PowerShell." Accessed: Nov. 26, 2025. [Online]. Available: https://learn.microsoft.com/en-us/powershell/module/microsoft.powershell.core/about/about_comparison_operators?view=powershell-7.5

[2]

"balenaEtcher - Flash OS images to SD cards & USB drives." Accessed: Nov. 26, 2025. [Online]. Available: https://www.balena.io/etcher

[3]

"C - NixOS Wiki." Accessed: Dec. 14, 2025. [Online]. Available: https://nixos.wiki/wiki/C

[4]

"curl." Accessed: Dec. 15, 2025. [Online]. Available: https://curl.se/

[5]

"fish shell." Accessed: Nov. 26, 2025. [Online]. Available: https://fishshell.com/

[6]

"Index of /pub/software/scm/git/." Accessed: Dec. 15, 2025. [Online]. Available: https://www.kernel.org/pub/software/scm/git/

[7]

"NixOS Manual." Accessed: Nov. 26, 2025. [Online]. Available: https://nixos.org/manual/nixos/stable/

[8]

"NixOS Search - Options." Accessed: Nov. 26, 2025. [Online]. Available: https://search.nixos.org/options

[9]

"NixOS Search - Packages." Accessed: Nov. 26, 2025. [Online]. Available: https://search.nixos.org/packages

[10]

“Nixos-rebuild - NixOS Wiki.” Accessed: Dec. 15, 2025. [Online]. Available: https://nixos.wiki/wiki/Nixos-rebuild

[11]

“Symlink Tutorial in Linux – How to Create and Remove a Symbolic Link,” freeCodeCamp.org. Accessed: Dec. 15, 2025. [Online]. Available: https://www.freecodecamp.org/news/symlink-tutorial-in-linux-how-to-create-and-remove-a-symbolic-link/

[12]

“Verifying Apache Software Foundation Releases | Apache Software Foundations.” Accessed: Nov. 26, 2025. [Online]. Available: https://apache.org/info/verification.html

[13]

D. McKay, “What is a TTY on Linux? (and How to Use the tty Command),” How-To Geek. Accessed: Nov. 26, 2025. [Online]. Available: https://www.howtogeek.com/428174/what-is-a-tty-on-linux-and-how-to-use-the-tty-command/

[14]

“wpa_supplicant.conf(5) - Linux man page.” Accessed: Nov. 26, 2025. [Online]. Available: https://linux.die.net/man/5/wpa_supplicant.conf

Appendix

Answers to questions

1) If I do an ad-hoc package install using nix-env, does a package entry get added automatically to the Nix config? How about if I do an apt install of a package?

   A package entry does not automatically get added to the Nix config file. Any package installed through ad-hoc methods are visible in the nix-environment and globally to all users on the system. It is independent of declarative management:

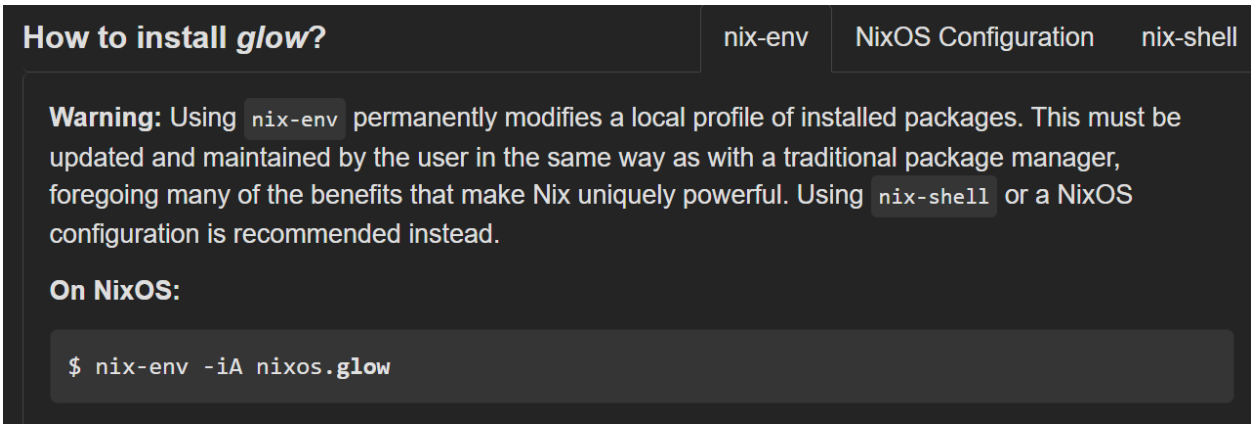

   Figure 1. Screenshot of glow package from Nix package search. Note that nix-env must be used in the same way as a traditional package manager, and does not have the benefits of Nix's unique declarative system.

   $nix-env -iA nixos.glow (get glow)
   $cd / ; sudo nano /etc/nixos/configuration.nix (package nixos.glow is not there)
   $nixos-rebuild switch (does nothing)

   Apt is not on the NixOS system by default, since Nix package management is there by default. I tried adding it to the system by adding:
   >pkgs.apt
   to the nix configuration file, and then rebuilding the system.
   Unfortunately, I could not get apt to work, as there seems to be some technical quirks to get it running in NixOS. Apt was originally dependent on Debian and Ubuntu-based systems, so it is not straightforward to set up on Nix. I would have to explore this question in future work.

2) If a package is installed manually from source into the usual location (/usr/local), is that package outside of Nix control?

To install from source, I first had to add curl to the Nix system by adding this line in the configuration.nix file:

- pkgs.curl

Curl will allow us to download files over protocols like https. So let's answer this question by installing git to the system.

I run curl to download this file:

$ cd ~

$ curl https://www.kernel.org/pub/software/scm/git/git-2.52.0.tar.gz

I then run these commands:

$ tar tvf git-2.52.0.tar.gz | less

$ mkdir git

$ mv git-2.52.0.tar.gz ./git/

$ cd git

$ tar xvf git-2.52.0.tar.gz

These commands view the contents of the latest git archive, place it into a new directory called 'git', and extracts the archive from within this new directory.

After running

$ cd git-2.52.0.tar.gz

I inspect the contents of the INSTALL file, and determine that I will be setting up git with default configuration (the program files will end up in /usr/local/ that way).

Thus:

$ less INSTALL

$ ./configure –help

$ ./configure

Should setup my program for make and install, but there is a problem: I do not have a gcc compiler for the C code in the source code. So I tried adding gcc to the nix environment, but it would not work…

$nix-env -iA nixos.libgcc

$ gcc (I get an error, command not found)

It turns out that C compiling and linking is completely different on a NixOS system. So I cannot just add a compiler to the packages environment and expect it to work. So the question of building form source on NixOS is one that will need further research into C compiling on Nix systems.

3) If I make a system service configuration change, e.g., by manually editing the ssh daemon configuration (/etc/ssh/sshd_config), how do I make sure that the Nix config reflects that change?

   I will answer this question with wpa_supplicant, which launches as a daemon during system initialization (to find registered WiFi networks) and also has a configuration file (/etc/wpa_supplicant.conf). When I registered my home network in this file, it was not present in /etc/nixos/configuration.nix. The only related line is where I enabled the network service wpa_supplicant. I can check the value of this variable in the Nix expression:

   $ nixos-option networking.wireless.enable

   To which the output value is true, since I have enabled wpa_supplicant. Note that the value is not my home wifi network information. This question could be compared to question 1. It is similar to ad-hoc package management, where changes are saved onto the system. So when we have wpa_supplicant, the changes exist independent of the configuration. We must manage app configuration on an app-by-app basis. But if we were to uninstall wpa_supplicant, by setting the networking.wireless.enable option to false, then we can assume that my home-network configuration would be wiped.

   So the answer is that you would have to copy the sshd or wpa_supplicant config file separately if you were to move to another machine, because the configuration.nix file is only responsible for system-wide functionality and package availability.

4) For user accounts, does the Nix config include things like the user information in /etc/passwd (preferred shell) and /etc/shadow (password hash), so that if I replicate the Nix setup on a different machine, the user accounts are fully replicated also?

   To try this, I will try updating my password, rebuilding the nix system, rebooting, and then launching from the *previous* configuration in the grub menu. This will test to see if my new password exists in the older version of the system. If it is there, then password is independent of Nix configuration.

After this experiment, the newer password is valid despite the system using a configuration before the new password was configured. This means that you cannot replicate passwords across machines, and this must be handled on a machine-by-machine basis. The only thing that can be configured are the user account names and what groups they belong to, as discussed earlier in the paper.

5) (Kind of related to Q3) Is there a Nix utility that automatically generates a Nix config from the current state of the system?

   In the /etc/nixos/configuration.nix file, there is an option for storing a copy of the current nix configuration:
   system.copySystemConfiguration
   That when set to true, will store a copy of the current system configuration in /run/current-system/configuration.nix. This is a handy backup in case if we lose the main configuration file. There is no 'automatic' configuration update daemon in NixOS. Having autosave-like configuration behavior, would be kind of like taking system snapshots every 5 minutes, and would likely result in too large of a performance degradation to reasonably implement.